Questions on Introduction to Class and Objects, April 15, 2026

Contents

1 Topics Discussed **2**

1.1 Approach 1: Put Declaration and Definition of a Class and Code to Test the Class in One File . . . 2

1.2 Approach 2: Separating Declaration, Definition, and Test Code . . . 4

1.2.1 FuelTank.hpp . . . 5

1.2.2 FuelTank.cpp . . . 5

1.2.3 TestFuelTank.cpp . . . 6

1.3 Approach 3: Separating Declaration, Definition, and Test Code; Use makefile . . . 7

1.3.1 makefile . . . 7

2 const for class method **8**

3 Constructors **8**

4 FuelTank Instance **9**

5 private vs. public **10**

6 Why would I need private classes? Don't we always need access to all things? **10**

7 So classes can come from different files and be in the same file, depending on how it's written. **10**

8 What do #ifndef and #define do? How different are these from #include? **11**

9 Is it possible to access private data members of a class, or how do we use them, and what is their purpose? **11**

10 What is .hpp? **12**

11 Does it matter if you define functions in the class or outside it? **12**

12 Can we reallocate the pointe "p" after setting it to be nullptr? **12**

13 How will class help with later topics and programming exercises? **12**

14 I don't understand the instructions for this. **13**

1 Topics Discussed

1.1 Approach 1: Put Declaration and Definition of a Class and Code to Test the Class in One File

Single File Convenience: This approach works well for small examples or test programs where the class is not intended for use by other source files in a larger project.

Best Practices: For larger projects, the industry standard practice is to place the class declaration in a .h (header) file and the member function definitions in a separate .cpp (source) file to improve compilation speed and organization.

Compilation: The compiler processes the single .cpp file, which contains everything needed to define the class and run the main function.

The following is complete code to define FuelTank class and use it.

code link: https://onlinegdb.com/DKXkywSaS5

```
//code link: https://onlinegdb.com/DKXkywSaS5

//Sample output:
//gas level of an empty fuel tank is 0
//add 5 gallons to tank
//send 3 gallons out from tank
//current gas level is 2

#include <iostream>

//Define FuelTank class.
//Methods are declared, but not defined (aka implemented).
class FuelTank {
public:
    FuelTank();
    FuelTank(int startLevel);
    void fillGas(int gallons);
    void sendGas(int gallons);
    int getGasLevel() const;

private:
    int currGasLevel;
};

//implement default constructor of FuelTank class,
//set data member currGasLevel to be 0.
FuelTank::FuelTank() {
    currGasLevel = 0;
    //int currGasLevel = 0;
```

```
    //WRONG, with int currGasLevel,
    //currGasLevel is a LOCAL variable in FuelTank,
    //NOT data member currGasLevel of FuelTank class.
}

//implement non-default constructor of FuelTank class,
//if formal parameter startLevel is larger than equal to zero,
//use it to initialize data member currGasLevel,
//otherwise, set data member currGasLevel to be 0.
//Rule of thumb: a constructor is to initialize data members,
//make sure that data members start with correct values.
FuelTank::FuelTank(int startLevel) {
    if (startLevel >= 0) {
       currGasLevel = startLevel;
    }
    else {
        currGasLevel = 0;
    }
}

//implement memember function fillGas,
//if given formal parameter gallons is non-negative,
//increase data member currGasLevel by gallons.
//otherwise, do we need to do anything?
//For simplicity, we do not consider capacity.
void FuelTank::fillGas(int gallons) {
    if (gallons >= 0) {
       currGasLevel += gallons;
    }
}

//implement member function sendGas sending gallons gas from fuel tank to engine.
//Given formal parameter gallons,
//if data member currGasLevel is at least gallons,
//subtract gallons from currGasLevel.
void FuelTank::sendGas(int gallons) {
     if (currGasLevel >= gallons) {
        currGasLevel -= gallons;
        //currGasLevel = currGasLevel - gallons;
     }
}

//implement member function getGasLevel returning the current value of data member
    currGasLevel.
//No data member is changed.
int FuelTank::getGasLevel() const {
```

```
    return currGasLevel;
}

int main() {
    //Call default constructor of FuelTank class to instantiate an object called tank
    //related: call default constructor of string class to instantiate an empty string object called str.
    FuelTank tank(0); //same as FuelTank tank; //same as FuelTank tank; //same as FuelTank tank; //same as FuelTank tank;
    //similar to
    //string str; //str is an empty string

    //print out the current gas level
    std::cout << "gas level of an empty fuel tank is "
        << tank.getGasLevel() << std::endl; //fill in ... part

    //write a statement to add 5 gallons of gas to tank
    //related: call substr method of str object to get a substring
    std::cout << "add 5 gallons to tank" << std::endl;
    tank.fillGas(5);

    std::cout << "send 3 gallons out from tank" << std::endl;

    //write a statement to send 3 gallons of gas from tank
    tank.sendGas(3);

    std::cout << "current gas level is ";
    //write a statement to find out and print the current gas level of tank
    //related: find out the length of a string object str
    std::cout << tank.getGasLevel() << std::endl;

    return 0;
}
```

1.2 Approach 2: Separating Declaration, Definition, and Test Code

For robust software engineering practices, especially in large projects, the standard C++ approach is to divide the code into three distinct files:

Header File (.h or .hpp): Contains the declaration of the class (the class blueprint, member variables, and function prototypes).

Source File (.cpp): Contains the definitions (implementation logic) of the class's member functions.

Test File (.cpp): Contains the main() function and the code dedicated to testing the functionality of the class.

This separation improves compile times (only changed implementation files need recompilation), enhances code organization, and facilitates code reuse across different parts of a project.

Code link: https://onlinegdb.com/MzXcjYT2S-

1.2.1 FuelTank.hpp

```
#ifndef FUEL_TANK_H
#define FUEL_TANK_H
class FuelTank {
public:
    FuelTank();
    FuelTank(int startLevel);
    int getGasLevel() const;
    void fillGas(int numGallons);
    void sendGas(int numGallons);

private:
    int currGasLevel;
};
#endif
```

1.2.2 FuelTank.cpp

```
#include "FuelTank.hpp"

FuelTank::FuelTank() {
   //int currGasLevel = 0; //wrong, with int before currGasLevel, the currGasLevel is
      a local variable
   //for default constructor,
   //that statement does not initialize data member currGasLevel
   currGasLevel = 0;
}

FuelTank::FuelTank(int startGasLevel) {
    if (startGasLevel >= 0) {
       currGasLevel = startGasLevel;
    }
    else {
       currGasLevel = 0;
    }
}

int FuelTank::getGasLevel() const {
    return currGasLevel;
}

void FuelTank::sendGas(int gallons) {
```

```
    if (currGasLevel >= gallons) {
       currGasLevel -= gallons;
    }
}

void FuelTank::fillGas(int gallons) {
    currGasLevel += gallons;
}
```

1.2.3 TestFuelTank.cpp

```
//Sample output:
//gas level of an empty fuel tank is 0
//add 5 gallons to tank
//send 3 gallons out from tank
//current gas level is 2

#include "FuelTank.hpp"
#include <iostream>

int main() {
    //TODO: Call default constructor of FuelTank class to instantiate an object called tank
    //related: call default constructor of string class to instantiate an empty string object called str.
    FuelTank tank(0); //same as FuelTank tank; //same as FuelTank tank; //same as FuelTank tank; //same as FuelTank tank;
    //similar to
    //string str; //str is an empty string

    //TODO: print out the current gas level
    std::cout << "gas level of an empty fuel tank is "
        << tank.getGasLevel() << std::endl; //fill in ... part

    //TODO: write a statement to add 5 gallons of gas to tank
    //related: call substr method of str object to get a substring
    std::cout << "add 5 gallons to tank" << std::endl;
    tank.fillGas(5);

    std::cout << "send 3 gallons out from tank" << std::endl;

    //TODO: write a statement to send 3 gallons of gas from tank
    tank.sendGas(3);

    std::cout << "current gas level is ";
```

```
    //TODO: write a statement to find out and print the current gas level of tank
    //related: find out the length of a string object str
    std::cout << tank.getGasLevel() << std::endl;

    //WRONG: currGasLevel is a private data member and cannot be accessed directly
        from a function outside the class where currGasLevel is defined
    //std::cout << tank2.currGasLevel << std::endl;
    return 0;
}
```

To run the code,

```
g++ -o run FuelTank.cpp TestFuelTank.cpp
./run
```

1.3 Approach 3: Separating Declaration, Definition, and Test Code; Use makefile

A `makefile` is a plain text file that contains instructions used by the make utility to automate software compilation and other tasks. It defines targets, dependencies, and shell commands (recipes) needed to build a project efficiently.

1.3.1 makefile

```
# This is an example Makefile for FuelTank project. This
# Typing 'make' or 'make run' will create the executable file.
#

# define some Makefile variables for the compiler and compiler flags
# to use Makefile variables later in the Makefile: $()
#
#  -g    adds debugging information to the executable file
#  -Wall turns on most, but not all, compiler warnings
#
# for C++ define CC = g++
CC = g++ -std=c++11
#CFLAGS = -g -Wall

# typing 'make' will invoke the first target entry in the file
# (in this case the default target entry)
# you can name this target entry anything, but "default" or "all"
# are the most commonly used names by convention
#
all: run

# To create the executable file run we need the object files
```

```
23 run:  TestFuelTank.o FuelTank.o
24     $(CC) -o run TestFuelTank.o FuelTank.o
25
26
27 # typing 'make' will invoke the first target entry in the file
28 # (in this case the default target entry)
29 # you can name this target entry anything, but "default" or "all"
30 TestFuelTank.o: TestFuelTank.cpp
31     $(CC) -c TestFuelTank.cpp
32
33 # To create the object file FuelTank.o, need source file FuelTank.cpp
34 FuelTank.o: FuelTank.cpp
35     $(CC) -c FuelTank.cpp
36
37 clean:
38     $(RM) run *.o *~
```

To use makefile, just type

```
make
```

From `all: run` of Line 20 of the above make file, we notice that the runnable file is called `run`. Then run the project using

```
./run
```

When you want to delete all the temporary files generated during the build process (such as object files .o, intermediate data files, and the final executable itself). Run the following command.

```
make clean
```

2 const for class method

What is the point of using const in `getGasLevel` method?

Answer: if we put const in `getGasLevel` method as shown in

```
int getGasLevel() const;
```

In case we update data members in `getGasLevel` method, compiler would complain. Reason: with `const` declaration, the method can only access – aka, perform read-only operations on – data members, it cannot change any data member of the class.

3 Constructors

Are there other specialized types of constructors besides copy and move that are commonly encountered?

Answer: Yes, besides copy and move constructors, several other specialized types of constructors are commonly encountered in C++. These include:

Default Constructor
Parameterized Constructor
Conversion Constructor
Delegating Constructor
Explicit Constructor
For more details, type the problem in google AI.
You may read the documents in constructors of `string` class.

4 FuelTank Instance

I can see that fuelTank instance in main is statically allocated, but I'm wondering when an object would be dynamically allocated since the class can implement dynamic allocation within?

Answer: The class itself doesn't control how an instance of it is allocated in a calling function (like main). An object is dynamically allocated when you explicitly request memory from the heap using new (in C++), rather than declaring it directly on the stack or as a global variable.

Here is when you would typically use dynamic allocation:

Variable Lifetime: When you need the object to persist beyond the scope in which it was created (e.g., a function returns an object that must still be valid later).

Large Objects: To avoid stack overflow errors when dealing with very large objects that consume significant memory.

Polymorphism (Inheritance): When you work with base class pointers or references to manage derived class objects, which is essential for virtual functions to work correctly.

Flexible Sizing (Containers): Classes like std::vector manage their internal storage dynamically on the heap to grow or shrink as needed, but the container instance itself might be statically allocated.

Statically allocated on the stack

```cpp
#include "FuelTank.h" // Assuming this header exists

int main() {
    FuelTank myTank(100); // Statically allocated instance
    // use myTank
    return 0;
}
```

Dynamically allocated on the heap

```cpp
#include <memory> // Recommended for modern C++

int main() {
    // Dynamically allocated using a raw pointer (requires manual delete)
    FuelTank* tankPtr = new FuelTank(100);
    // use tankPtr->fill(50);
    delete tankPtr; // MUST delete manually to avoid memory leak

    // Dynamically allocated using a smart pointer (recommended)
    auto smartTankPtr = std::make_unique<FuelTank>(100);
    // use smartTankPtr->fill(50);
```

```
    // memory is freed automatically when smartTankPtr goes out of scope

    return 0;
}
```

The key difference is where the memory resides and who is responsible for managing its lifespan. The code within the FuelTank class remains the same regardless of how its instance is created.

The above examples come from google AI.

5 private vs. public

Please explain more details on private and public class and the process of declaring methods in the class.

Answer: data members and methods declared in `private:` section of a class cannot be accessed by the functions outside the class where these data members and methods reside.

6 Why would I need private classes? Don't we always need access to all things?

You are likely referring to private data members, as "private classes" are a more advanced topic we haven't covered yet. Setting data members to private is a practice called encapsulation. It ensures data integrity by preventing outside code from accidentally or maliciously corrupting an object's internal state. To interact with this private data safely, we use three specific tools:

- **Constructors:** These initialize the object's data members to a valid starting state when the object is first created.
- **Accessors (Getters):** These allow other parts of the program to read the current value of a data member without the risk of modifying it.
- **Modifiers (Setters):** These allow controlled changes to data members. Unlike direct access, a setter can include logic to "validate" data (e.g., ensuring a "price" variable is never set to a negative number).

7 So classes can come from different files and be in the same file, depending on how it's written.

Answer: Yes. While small programs can stay in one file, professional software is usually organized into separate files to make it easier to manage and reuse. Think of it like a manufacturing process:

- **The Header File (.h):** This is the blueprint. It documents what the class "looks like" and what it can do, without showing the complicated inner workings.
- **The Class Definition (.cpp):** This is the factory. This file contains the actual source code that carries out the instructions defined in the blueprint.

- **The Main/Test File (.cpp):** This is the consumer. This file "buys" or uses the finished product to perform tasks or run tests.

For a practical look at how these files link together, please refer to the examples in Section 1.2.

8 What do #ifndef and #define do? How different are these from #include?

While `#include` tells the compiler to copy-paste the contents of a file, `#ifndef` and `#define` act as an Include Guard. Their job is to ensure that a class is declared exactly once, even if its header file is included multiple times.

```
#ifndef FUEL_TANK_H
#define FUEL_TANK_H
// Class declaration for FuelTank goes here...
#endif
```

The guard checks if the unique label (`FUEL_TANK_H`) has been "defined" yet. If it has, the compiler skips the entire block of code, preventing duplication.

Why is this necessary? Imagine a scenario with inheritance:

1. Class **Animal** is defined in `animal.hpp`.
2. Class **Hare** inherits from Animal (so `hare.hpp` includes `animal.hpp`).
3. Class **Tortoise** also inherits from Animal (so `tortoise.hpp` also includes `animal.hpp`).

If you create a `competition.cpp` file that includes both `hare.hpp` and `tortoise.hpp`, the compiler will try to pull in `animal.hpp` twice. Without an include guard, the compiler sees two identical declarations for the **Animal** class, resulting in a "redefinition" error.

The Benefit:

Standard libraries (like `iostream`) already use these guards. This is why your code still compiles even if you accidentally include the same library twice:

```
#include <iostream>
#include <iostream> // Redundant, but harmless due to include guards
```

9 Is it possible to access private data members of a class, or how do we use them, and what is their purpose?

Answer: You cannot access private data members directly from outside the class (e.g., using object.variable). Instead, you interact with them through public "interface" methods.

- **Accessing (Getters):** Use "getter" methods to retrieve the value. For example, in the `std::string` class, the actual character array is private, but you can find its length using the `.length()` or `.size()` methods.

- **Modifying (Setters):** Use "setter" methods or operators to change the value. In the `string` class, you can modify the private data using the `+` concatenation operator.

The Purpose: Data Protection (Encapsulation) The goal of making data private is to ensure the object stays in a valid state. By forcing access through methods, the class acts as a "gatekeeper." For instance, a `Date` class can have a setter for `month` that prevents a user from accidentally setting it to 13 or -5. If the variable were public, there would be no way to stop such errors.

10 What is .hpp?

Header File (.h or .hpp): Contains the declaration of the class (the class blueprint, member variables, and function prototypes).

11 Does it matter if you define functions in the class or outside it?

Functions defined inside a class are called methods (or member functions). They typically represent operations on the class's data and have direct access to its internal members without needing them passed as parameters. Because they operate directly on the object's state, they often don't need to return modified data explicitly.

In contrast, functions defined outside a class cannot access private or protected members. They must interact with an object through its public interface, usually by calling methods using the syntax `object.methodName(parameters);`.

12 Can we reallocate the pointe "p" after setting it to be nullptr?

Answer: Yes, but it is important to distinguish between the pointer and the address it holds.

Once you set a pointer to nullptr, it no longer points to a valid memory location. Therefore, it cannot be dereferenced (you cannot use *p to access or modify a value), as doing so would cause your program to crash. However, you can absolutely reuse the pointer variable p. Since p is essentially a variable that stores an address, you can assign it a new valid address at any time. For example:

- **Resetting:** `p = &anotherVariable;` (Points `p` to an existing object).
- **Reallocating:** `p = new int[10];` (Points `p` to a newly allocated block of memory).

In short: while you can't use the "nothing" that nullptr represents, you can always give the pointer a new "something" to point to.

13 How will class help with later topics and programming exercises?

Pleaes refer to Project 3: Turtle Graphics, Lab 10 and Lab 11.

14 I don't understand the instructions for this.

Please watch the lecture video and answer to the lecture slip on Brightspace. To find the correct slip, navigate to Content and search for the specific date of the class.

If you need further assistance, I can provide more details on navigating Brightspace or locating specific files.

15 I am still a bit confused about how to access the different classes.

We will work on Project 3: Turtle Graphics.